

Personal OS Build Cheat Sheet

How to build your own AI-native operating system, from scratch, in 2 weeks of evenings.

By Miles Deutscher / AI Edge

How to use this PDF

Drop this entire PDF into Claude Code, Codex, Cursor agents, or Aider — it will read every section and walk you through the build, step by step. Each section has a copy-paste prompt block (dark background) that you can hand directly to your coding agent. The agent does the typing; you make the decisions.

You don't have to copy my stack. Every major decision in the build has alternatives listed — Supabase or Firebase, Claude or Codex, Anthropic or OpenAI, Vercel or self-host. Pick the parts that match your taste and your budget. The system architecture is the same either way.

Reading order: Part 1 → Part 7 in sequence the first time. After that this is a reference — jump to the section you're building.

What you'll have at the end

- A live dashboard at your-domain.vercel.app with seven AI-powered cards.
- A Telegram bot that captures voice notes from anywhere and routes them into the right database table — automatically.
- A finance card that reads your messy Google Sheet and figures out your net worth without you labeling anything.
- A journal that summarises your day in 5 seconds.
- A habits tracker, calendar, goals card, and CRM that all sync across phone and laptop in real time.
- Total cost: ~\$30/month at active use. Time saved: at least an hour a day.

What you need before you start

- **An AI coding agent.** Claude Code (recommended), Codex, Cursor with agent mode, or Aider. Any of them work.

- **An Anthropic or OpenAI API key.** Both work; the system supports either as primary with the other as fallback.
- **A Supabase account** (free tier is fine). Or Firebase, Turso, Neon — see Part 3 for alternatives.
- **A Vercel account** (free tier is fine). Or Netlify, Fly.io, Railway, self-host.
- **A Telegram account.** For the voice-capture bot. Optional but strongly recommended.
- **~10–15 hours of focus time.** Spread over 2 weeks of evenings, or one heavy weekend.

Contents

Part 1	Decide your stack	p. 3
Part 2	Design the dashboard before you build it	p. 5
Part 3	Foundation — Next.js + database + auth	p. 6
Part 4	Capture pipeline — voice → AI → database	p. 9
Part 5	The seven cards — build each surface	p. 12
Part 6	Memory — the brain layer	p. 19
Part 7	Deployment + cron + privacy	p. 21
Part 8	Common bugs and how to debug them	p. 23
Part 9	Going further	p. 25
Appendix A	All copy-paste prompts in one place	p. 26
Appendix B	Env vars checklist	p. 28

Part 1 · Decide your stack

Before you write a line of code. Pick once, change rarely.

Frontend framework

You need a framework that can serve a dashboard AND run server-side AI calls. The dashboard is mostly client-side React; the AI work is server-side functions.

DEFAULT · Next.js 15 (App Router)

- **Why:** first-class on Vercel. Server components + serverless functions in the same project. Free tier covers personal use.

ALTERNATIVES

- **SvelteKit** — same model, smaller bundle, less mature ecosystem.
- **Remix** — solid choice if you prefer their data-loading pattern.
- **Astro** — only if your dashboard is mostly static.
- **Plain Express + React** — works, but you'll be writing infrastructure plumbing the frameworks above give you free.

Database

DEFAULT · Supabase (Postgres + pgvector)

- **Why:** Postgres with vector embeddings built in. Free tier covers personal use. RLS policies. JSON columns when you need them. SQL when you don't.

ALTERNATIVES

- **Firebase** — Google's offering. Easier auth, harder vector search. Good if you don't need semantic memory.
- **Turso** — SQLite at the edge. Cheap, fast for reads. No vector support — pair with Pinecone or supermemory if you go this route.
- **Neon** — serverless Postgres. Very similar feel to Supabase. Slightly less batteries-included.
- **PlanetScale** — MySQL. Great for high-write workloads. Overkill for personal use.

LLM provider

DEFAULT · Anthropic Claude (primary) + OpenAI (fallback)

- **Why:** Claude wins on writing quality and structured output. OpenAI wins on cheap, fast classification + Whisper transcription. Use both, one as primary, one as fallback.

ALTERNATIVES

- **OpenAI alone** — fine if you don't want two API keys to manage.
- **Google Gemini** — generous free tier, weaker structured output.
- **Local Llama** — full privacy, but you need a beefy machine and you'll lose the speed.

Coding agent

DEFAULT · Claude Code

- **Why:** reads your codebase. Edits files in place. Runs commands. Best at long-form architecture decisions.

ALTERNATIVES

- **Codex (OpenAI)** — works similarly. Cheaper per token. Slightly weaker on multi-file refactors.
- **Cursor (with agent mode)** — IDE-native. Good if you prefer typing alongside the agent.
- **Aider** — terminal-only. Best for small, surgical edits.

Hosting

DEFAULT · Vercel

- **Why:** first-class Next.js support. Cron jobs included on free tier. Env vars + secrets handled cleanly. ~\$0/month for personal use.

ALTERNATIVES

- **Netlify** — very similar. Slightly fewer Next.js niceties.
- **Fly.io** — full Docker control. Choose this if you need a long-running process (e.g. local Llama).
- **Railway** — simple, beautiful. ~\$5/month minimum.
- **Self-host on a VPS** — full control, more work. Hetzner / OVH for cheap hardware.

Capture surface

DEFAULT · Telegram bot

- **Why:** push voice + text to a webhook in 3 lines. No app to build. Works on every phone. Free.

ALTERNATIVES

- **WhatsApp Business API** — better if your contacts already use it. More setup, slightly less flexible.
- **Discord bot** — same model. Use if you live in Discord.
- **Native iOS Shortcut** — great UX, only works on Apple. Pairs well with the dashboard's web capture form.
- **Just the dashboard's web form** — no bot at all. Open the dashboard on your phone and type. Lose the voice-from-anywhere magic.

Part 2 · Design the dashboard before you build it

I designed the dashboard in Claude Design before writing any code. Worth the 30 minutes.

Why design first

Dashboards are visual problems before they're engineering problems. Designing first means you discover layout issues, colour problems, and card-density issues while it's a 30-minute fix in a design tool — not a 6-hour rebuild later. It also gives Claude Code a concrete target to copy from in Part 3.

What I used · Claude Design

Claude Design lets you generate full HTML/CSS layouts conversationally. I described the dashboard in plain English — “a personal-OS-style dashboard with a top rail, a 3-column grid, glassmorphism cards, monospace accents” — and got back a working HTML file with all seven cards laid out. Tweaked it for an hour, then moved to engineering.

PROMPT · paste into your AI coding agent

I'm building a personal AI dashboard. Generate a single-page HTML mockup with this layout:

- Top rail: brand on the left, tabs in the centre (Home, CRM, Brain, Finance, Journal, Health), date/time + avatar on the right.
- Below the rail: a 3-column grid (left narrow, centre wide, right narrow).
- Left column: 3 stacked cards — Operator, Finance Pulse, Key Blockers.
- Centre column: 3 stacked cards — Session, Habit Tracker, Priorities.
- Right column: 1 card — Nutrition.
- Style: dark mode, glassmorphism, oklch colour tokens, monospace numbers, thin borders, backdrop blur on cards.

Make each card visually distinct but cohesive. Use placeholder data.

Alternatives if you don't use Claude Design

- **Figma** — the classic. More control, slower iteration.
- **v0.dev** — Vercel's component generator. Tailwind-native, pairs well with Next.js.
- **Just sketch on paper** — for small projects this is genuinely fine. You don't need pixel-perfect to start.
- **Skip design entirely** — go straight to Claude Code with a written description. Slower iteration but works.

What to lock before moving on

- Number and shape of cards on the home page (mine has 7).
- How many tabs / pages (mine has 6: Home, CRM, Brain, Finance, Journal, Health).
- Colour palette — pick 1 accent colour, 4 shades of neutral, semantic OK/warn/danger.
- Typography — display font + monospace font for numbers.
- Whether to support a demo mode for video / sharing (recommended).

Part 3 · Foundation

Next.js project + Supabase + auth gate. The minimum viable shell to build everything else on.

Step 1 · Spin up the Next.js project

PROMPT · paste into your AI coding agent

```
Create a new Next.js 15 project with the App Router and TypeScript strict mode. Set up Tailwind CSS with a dark default theme. Add a basic globals.css with oklch colour tokens (ink-0 through ink-4 for neutral, one accent, semantic ok/warn/danger). Initialise git. Don't scaffold pages yet – just the empty shell.
```

Step 2 · Set up Supabase (or alternative)

Create a project at supabase.com. From the dashboard, grab three values:

- Project URL → goes into NEXT_PUBLIC_SUPABASE_URL
- Anon key → goes into NEXT_PUBLIC_SUPABASE_ANON_KEY
- Service role key → goes into SUPABASE_SERVICE_ROLE_KEY

Enable the vector extension under Database → Extensions. You'll need it for memory in Part 6.

Step 3 · Write the schema

PROMPT · paste into your AI coding agent

```
Create a Supabase migration that adds these tables:
```

```
- entities: id, user_id, name, kind, metadata (jsonb), created_at
- raw_captures: id, user_id, source, raw_text, audio_url, classification (jsonb), llm_source, routed_to, routed_id, created_at
- tasks: id, user_id, title, description, urgency, key (boolean), priority_score, time_estimate_min, tags (text[]), due_date, owner, entity_id (FK), completed_at, created_at, updated_at
- daily_logs: id, user_id, log_date (date NOT NULL), notes (text – will hold JSON for habits/nutrition/finance/goals), mood, created_at, updated_at
- memory_chunks: id, user_id, source_type, source_id, text, embedding (vector(1536)), created_at
- audit_log: id, user_id, action, resource_type, resource_id, metadata (jsonb), created_at
```

```
Add an ivfflat index on memory_chunks.embedding using vector_cosine_ops. Enable RLS on all tables with deny-all policies (service role bypasses RLS). Save as supabase/migrations/0001_init.sql.
```

Step 4 · Add the auth gate

For a single-user personal dashboard, full OAuth is overkill. A signed cookie gated by a single password works fine.

PROMPT · paste into your AI coding agent

Add a single-password auth gate to the Next.js app. Use HMAC-signed cookies. Env vars: AUTH_SECRET (random 32-byte hex) and DASHBOARD_PASSWORD (memorable string).

Create:

- middleware.ts – gates all routes except /login, /api/auth/*, and any webhook routes.
- app/login/page.tsx – single password field, posts to /api/auth/login.
- app/api/auth/login/route.ts – verifies password, sets signed cookie.
- app/api/auth/logout/route.ts – clears the cookie.

API routes can also accept an x-api-secret header for programmatic access (CLI, cron, etc.).

Step 5 · Port the design from Part 2

PROMPT · paste into your AI coding agent

I have a Claude Design HTML mockup at [paste path / paste HTML inline]. Port this into Next.js components under components/dashboard/:

- One component per card.
- A shared Panel wrapper for the glassmorphism style.
- A TopRail component for the header.
- A Shell component for the page layout.

Wire the layout but use placeholder data inside each card for now – we'll connect data in later parts.

Part 4 · Capture pipeline

The single most important feature. Voice → AI → database. Build this BEFORE the dashboard cards — every other feature reads from what this pipeline writes.

How it flows

Voice note hits Telegram bot → webhook fires → Vercel function picks it up → Whisper transcribes → Claude classifies (kind, urgency, entity, tags) → row written to the right table (tasks, journal, capture, etc.) → memory chunk embedded → audit log entry. End-to-end target: under 5 seconds.

Step 1 · Set up the Telegram bot

- In Telegram, search @BotFather. Send /newbot.
- Pick a display name (e.g. “My OS Capture”) and a username (must end in _bot).
- Save the bot token. → TELEGRAM_BOT_TOKEN
- Generate a webhook secret: openssl rand -hex 16. → TELEGRAM_WEBHOOK_SECRET
- Get your numeric Telegram user ID via @userinfobot. → TELEGRAM_USER_ID (so the bot only listens to you)

Step 2 · Build the webhook + classifier

PROMPT · paste into your AI coding agent

Build a Telegram webhook endpoint at `app/api/telegram/webhook/route.ts` that:

1. Verifies the `x-telegram-bot-api-secret-token` header matches `TELEGRAM_WEBHOOK_SECRET`.
2. Verifies the message is from `TELEGRAM_USER_ID`.
3. If voice: download the audio, transcribe with OpenAI Whisper.
4. Pass the text through a classifier in `lib/router/classifyCapture.ts` that returns `{ kind, urgency, entity_id, tags, summary }`. Use Anthropic Claude as primary, OpenAI as fallback, simple regex as last resort.
5. Write the raw capture to `raw_captures`.
6. Route to the right downstream table (tasks, decisions, notes) per kind.
7. Embed the text via OpenAI text-embedding-3-small and write to `memory_chunks`.
8. Write an `audit_log` row.
9. Reply to the user via Telegram with a confirmation + an inline urgency keyboard (Today / This Week / This Month / Someday / Key) so they can override the AI's urgency call with a tap.

Step 3 · Register the webhook with Telegram

```
curl -F "url=https://your-app.vercel.app/api/telegram/webhook" \\  
-F "secret_token=$TELEGRAM_WEBHOOK_SECRET" \\  
"https://api.telegram.org/bot$TELEGRAM_BOT_TOKEN/setWebhook"
```

Step 4 · Build the web fallback capture form

Telegram is amazing for voice, but sometimes you're at a desk and just want to type. Add a small floating capture box on the dashboard.

PROMPT · paste into your AI coding agent

Add a floating capture box that sits at the bottom of the dashboard. When focused, expands. Submitting POSTs to /api/capture which runs the same pipeline as the Telegram webhook (classify → write → embed → audit). Show a small toast on success.

Alternatives

If you don't want Telegram

- **iOS Shortcut + dashboard endpoint:** create a Shortcut that sends a voice memo (transcribed by Apple) to your /api/capture endpoint with an API secret. Same effect, no third-party app.
- **WhatsApp Business API:** functionally identical to Telegram but with more setup paperwork.
- **Just the web form:** lose the voice-from-anywhere magic but simpler. Pairs well with mobile browser shortcuts.

Common bugs in this section

- **Webhook returns 200 but nothing happens.** Check the bot is sending you the message — Telegram sometimes silently fails to deliver on its side. Use the bot's getUpdates endpoint to verify.
- **Whisper returns empty string.** The audio file format from Telegram is OGG; Whisper handles it but only with the right MIME type. Don't pass the wrong content-type.
- **Classifier returns hallucinated entity IDs.** Always validate returned IDs exist in your entities table before writing. Add a fallback to null if the AI invents one.

Part 5 · The seven cards

Each card is a self-contained component reading and writing through one or two API routes. Build them in this order — easiest first, hardest last.

5.1 · Operator + Session

Operator: your name, location, current focus. Pure UI, no backend. Session: today's top three tasks, ranked by AI. Reads from the CRM. Easiest card to build first because it just renders.

PROMPT · paste into your AI coding agent

Build the Operator + Session cards.

Operator: static. Renders my name, location, role, current focus from a config file.

Session: reads from `/api/tasks?status=open`, filters to urgency = today and key = true, ranks by priority_score, shows top 3. Each row has time estimate and click-through to the CRM.

5.2 · Calendar

Pulls from Google Calendar via the secret iCal URL (no OAuth required). Renders a 14-day week strip; click any day to view its events.

PROMPT · paste into your AI coding agent

Build a Calendar card. Read `GOOGLE_CALENDAR_ICAL_URL` from env. Use `ical.js` (NOT `node-ical` — see Part 8) to parse the feed. Expand recurring events using `event.iterator()` within a 14-day window. Render a 7-day week strip; clicking a day shows that day's events in detail. Auto-scroll to a NOW marker for today.

API route at `app/api/calendar/route.ts`. Cache the parsed feed in module memory for 5 minutes. Return `cache-control: no-store` on the response.

5.3 · Habit Tracker

Six daily checks. Click them off. Cross-device sync. Daily reset at midnight in YOUR timezone — not the server's.

PROMPT · paste into your AI coding agent

Build a Habit Tracker. Six configurable habits. State stored in `daily_logs.notes` as JSON: `{ habits: { done: string[], total: number } }`.

Use a `localStorage` cache for instant feedback. Sync to Supabase on every click via `POST /api/habits/[date]`. On mount, fetch `GET /api/habits?days=30` and merge with `localStorage`.

Critical: use a `localDateKey()` helper that uses the user's clock for "today", not the server's. Otherwise the day rolls over at the wrong time for non-UTC users.

5.4 • CRM (the biggest card)

Tasks, four urgency tiers, three views (Smart / Kanban / Category), drag-drop reorder, click-to-edit drawer, AI-assisted “smart search.”

PROMPT • paste into your AI coding agent

```
Build a full CRM card. Four urgency tiers: Today, This Week, This Month, Someday.
Each task has { id, title, description, urgency, key, priority_score, tags, due_date,
entity_id, owner, completed_at }.
```

Three views, switchable via tabs:

- **Kanban**: 4 columns by urgency. Drag-drop reorder within a tier; persist priority_score on drop.
- **Smart**: a search box that uses Claude to interpret natural language (“what should I do this morning”) and return matching task IDs.
- **Category**: tasks grouped by entity_id.

Click any task → side drawer for inline edit. New tasks insert at the top of their tier. Persist view selection to localStorage so refreshes don’t reset it.

```
API: GET /api/tasks?status=open|done, POST /api/tasks, PATCH /api/tasks/[id], DELETE
/api/tasks/[id], POST /api/tasks/smart (NL search via Claude).
```

5.5 • Nutrition

Type a meal. AI returns macros. Bidirectional coupling — edit any macro, kcal recomputes from the formula. Edit kcal, AI redistributes macros.

PROMPT • paste into your AI coding agent

```
Build a Nutrition card. State stored in daily_logs.notes JSON under nutrition.meals.
Each meal: { id, t, n, kcal, p, c, f, estimated }.
```

Two API routes:

- POST /api/nutrition/estimate takes { text }, returns { kcal, p, c, f } via Claude.
- POST /api/nutrition/redistribute takes { name, kcal }, returns { p, c, f } for that food at that calorie target.

Click any meal → inline editor. Edit any macro → $kcal = 4 \cdot p + 4 \cdot c + 9 \cdot f$, computed instantly. Edit kcal → debounced 600ms, then call /redistribute with the meal name + new kcal, get back fresh macros.

Daily reset: storage key is miles-nutrition-{YYYY-MM-DD} so the card auto-clears at local midnight.

5.6 • Health Tab

30-day calorie log table. Read-only aggregation from nutrition data.

PROMPT · paste into your AI coding agent

Build a Health tab as a separate page. Reads from GET /api/nutrition?days=30 and renders a table: one row per day, columns for KCAL / P / C / F / MEALS. Click a row to expand into the individual meals. Show averages (over logged days only – exclude zero-meal days) at the top.

5.7 · Goals

Weekly + monthly goal lists. Persistent — never auto-clear. User checks them off or removes them manually.

PROMPT · paste into your AI coding agent

Build a Goals card with two stacked sections: THIS WEEK and THIS MONTH. Each section has a list + an “add a goal” input.

Critical: store on a SENTINEL date (e.g. 2000-01-01) so goals never auto-clear at week/month boundaries. Schema: daily_logs.notes has goals_week_items: GoalItem[] and goals_month_items: GoalItem[] on that one row.

API: GET /api/goals, POST /api/goals with { scope: 'week' | 'month', items: GoalItem[] }.

5.8 · Finance Pulse (the AI showpiece)

Reads your messy Google Sheet. AI figures out net worth + categories. No labels needed. Daily auto-snapshot via cron.

PROMPT · paste into your AI coding agent

Build a Finance Pulse card with AI-driven multi-tab extraction.

Auth: Google service account (NOT publish-to-web – see Part 7). Env vars: GOOGLE_SHEETS_FINANCE_ID, GOOGLE_SERVICE_ACCOUNT_EMAIL, GOOGLE_SERVICE_ACCOUNT_KEY.

Pipeline: download workbook as XLSX via Drive API → parse all tabs with `exceljs` into 2D arrays → send dump to Claude with one instruction: “extract net_worth, currency, as_of, categories[]. Avoid double-counting. Output JSON only.” → persist snapshot to daily_logs.notes.finance.

Critical UX rule: page loads NEVER trigger AI. They read the latest snapshot from Supabase. ONLY a manual refresh button (?refresh=1) or a Vercel cron triggers the AI pipeline. Otherwise users will burn your API budget on every page view.

Daily snapshot: Vercel cron at vercel.json hits /api/finance/snapshot at 5am UTC daily. Auth via Authorization: Bearer \${CRON_SECRET}.

Part 6 · Memory — the brain layer

What lets you ask “what was that idea I had at the gym in March” and have AI find the right capture. The difference between a notebook and a brain.

Concept

Every text artifact you create — a capture, a task title, a journal entry, a completed item — gets converted to a vector embedding (a list of ~1500 floating-point numbers) and stored in a vector database. When you ask a question in natural language, you embed the question with the same model, then find the closest vectors via cosine similarity. Those are your relevant memories. Pass them to Claude as context and let it answer.

Default approach · Supabase pgvector + OpenAI embeddings

PROMPT · paste into your AI coding agent

Add a memory layer.

On every write to raw_captures, tasks, journal entries, completed tasks, or daily_logs.notes, also write a row to memory_chunks:

- source_type: capture | task | journal | habit | meal | etc.
- source_id: the original row's id
- text: the canonical text representation
- embedding: vector(1536) generated via OpenAI text-embedding-3-small

Add a search endpoint POST /api/memory/search that:

1. Embeds the query.
2. Runs `SELECT * FROM memory_chunks ORDER BY embedding <=> $1 LIMIT 20`.
3. Returns the top 20 with their original source rows joined in.

Add a Brain tab that uses this endpoint as a search bar. Type a question, see the top matches. Click any match to open its source.

Alternatives

Managed vector / memory services

- **supermemory.ai** — opinionated, simpler API. Stores text + does embedding + search for you. Good for non-engineers.
- **mem0** — open source, self-hosted memory layer with built-in reasoning over recall. Worth considering if you want richer memory behaviour.
- **Pinecone** — pure vector DB. Pair with your own embedding pipeline (Cohere, OpenAI, Voyage). Most flexible, most setup.
- **Weaviate** — open source, hybrid search (semantic + keyword) out of the box.
- **Chroma** — embeddable Python-first, fine for single-user.

Embedding model alternatives

- **OpenAI text-embedding-3-small** (default) — 1536 dims, ~\$0.02/M tokens. Cheap and fast.
- **OpenAI text-embedding-3-large** — 3072 dims, slightly higher quality, 4x cost. Worth it for technical or domain-specific text.
- **Voyage AI** — better than OpenAI on retrieval benchmarks. Pricier.
- **Cohere embed-v3** — solid, multilingual.
- **Local (sentence-transformers)** — free, slower, lower quality. Worth it if you don't want any text leaving your machine.

When to skip memory

If your dashboard is mostly “today” focused — habits, today's tasks, today's meals — you don't need vector search. Plain SQL queries (`WHERE created_at > X`) cover 80% of usage. Add memory later when you start asking questions about a YEAR of data, not a week.

Memory · the prompt

Once you have memory, you can build “ask my OS” — type a question, get an answer with citations:

PROMPT · paste into your AI coding agent

Add an `/ask` endpoint that:

1. Embeds the user's question.
2. Pulls top 20 most-similar `memory_chunks` via cosine similarity.
3. Joins to source rows for full context.
4. Sends the question + the 20 chunks (truncated to 200 tokens each) to Claude with this system prompt:

“You are the user's personal assistant. Answer the question using ONLY the context provided. Cite sources by referring to capture IDs in [brackets]. If you don't have enough context, say so.”

5. Streams the answer back to the UI.

Part 7 · Deployment + cron + privacy

Getting it live, keeping it running, keeping it private.

Step 1 · Push to Vercel

```
npm i -g vercel
cd your-personal-os
vercel link
vercel --prod
```

Then push every env var:

```
vercel env add NEXT_PUBLIC_SUPABASE_URL production
vercel env add NEXT_PUBLIC_SUPABASE_ANON_KEY production
vercel env add SUPABASE_SERVICE_ROLE_KEY production
vercel env add ANTHROPIC_API_KEY production
vercel env add OPENAI_API_KEY production
vercel env add AUTH_SECRET production
vercel env add DASHBOARD_PASSWORD production
vercel env add API_SECRET production
vercel env add TELEGRAM_BOT_TOKEN production
vercel env add TELEGRAM_WEBHOOK_SECRET production
vercel env add TELEGRAM_USER_ID production
vercel env add GOOGLE_CALENDAR_ICAL_URL production
vercel env add GOOGLE_SHEETS_FINANCE_ID production
vercel env add GOOGLE_SERVICE_ACCOUNT_EMAIL production
vercel env add GOOGLE_SERVICE_ACCOUNT_KEY production
vercel env add CRON_SECRET production
```

Step 2 · Set up the Vercel cron

Add a vercel.json at the repo root:

```
{
  "crons": [
    {
      "path": "/api/finance/snapshot",
      "schedule": "0 5 * * *"
    }
  ]
}
```

The cron endpoint must verify the Authorization: Bearer \${CRON_SECRET} header — Vercel sends it automatically.

Step 3 · Privacy — Google service account

For finance data especially, never use Google’s “publish to web” option. The URL has a noindex header but it’s still a public-but-unguessable link. Use a service account instead — sheet stays fully private.

- Go to console.cloud.google.com/projectcreate. Create a project.
- Enable the **Drive API** and **Sheets API**.

- Create a service account under IAM → Service Accounts.
- Generate a JSON key. Download it.
- Open your finance Google Sheet → Share → paste the service-account email (ends in @...iam.gserviceaccount.com) → Viewer.
- Push the email and the private_key field from the JSON to Vercel as GOOGLE_SERVICE_ACCOUNT_EMAIL and GOOGLE_SERVICE_ACCOUNT_KEY.

Step 4 · Point Telegram at the live webhook

```
curl -F "url=https://your-app.vercel.app/api/telegram/webhook" \\  
-F "secret_token=$TELEGRAM_WEBHOOK_SECRET" \\  
"https://api.telegram.org/bot$TELEGRAM_BOT_TOKEN/setWebhook"
```

Step 5 · Backup

Supabase auto-backs up daily on the free tier. For belt-and-suspenders, add an /api/admin/export endpoint that returns a JSON snapshot of every table. You can hit this from a cron or a button to download a manual backup whenever.

Part 8 · Common bugs and how to debug them

Real bugs from the build. Each one cost me hours. Read this BEFORE you hit them.

1 · “o.BigInt is not a function” on Vercel

Symptom: works locally, breaks on production deploy. Error mentions BigInt.

Cause: Next.js’s bundler tree-shakes some Node-native libraries (notably node-ical, rrule, anything using BigInt internally) and mangles the globals.

Fix: don’t use those libraries. Swap to a pure-JavaScript alternative. For iCal: use ical.js (Mozilla’s reference parser). For RRule expansion: also covered by ical.js. serverExternalPackages in next.config.mjs sometimes works, often doesn’t — don’t rely on it.

2 · Habits reset at 4am

Symptom: your dashboard resets habits before you go to bed.

Cause: the day rolls over at midnight UTC, not midnight in your timezone.

Fix: write a localDateKey() helper that uses the user’s clock. Use it everywhere you ask “what day is it?” — including the storage key and the day-anchor for daily_logs writes.

3 · Silent POST failures

Symptom: a card optimistically updates, then the change disappears a few seconds later. Refresh — gone.

Cause #1: The POST is returning 500 silently because your fetch error handler is empty (.catch() => {})). Always log errors.

Cause #2: A NOT NULL column missing from your INSERT. Especially common with shared upsert helpers — they work for tables where rows already exist (UPDATE branch) but break on first INSERT for a new key.

Fix: surface every server error to the console at minimum. Validate INSERTs have every NOT NULL column.

4 · Race: GET clobbers a fresh local edit

Symptom: user adds an item; it appears; then disappears even though the POST succeeded.

Cause: a mount-time GET that resolved AFTER the user’s first edit. The empty server response overwrites the local state.

Fix: use a dirtyRef set to true on first user edit. Once dirty, ignore the mount-time GET’s response.

5 · Stale Supabase reads

Symptom: you mark a task done, refresh, it’s undone again.

Cause: PostgREST’s edge cache serves stale snapshots for bulk SELECTs.

Fix: append a unique limit per request — `.limit(100000 + Date.now() % 100000)` — to bust the cache. Or use `.eq(...)` filters that are sufficiently unique that no two requests hit the same cache key.

6 • AI extracts the wrong number

Symptom: Claude returns a net worth that doesn't match your sheet.

Cause #1: it's double-counting. A summary tab plus per-category tabs.

Cause #2: it picked the wrong row from a daily-snapshot tab.

Fix: be explicit in the system prompt about “avoid double-counting” and “use only the most recent row of any time-series tab.” Add a notes field to the response schema where Claude can flag ambiguity for human review.

7 • Client crashes on first render

Symptom: page shows “Application error: a client-side exception has occurred.”

Cause: TypeScript's non-null assertion (!) lying to you. A field is null at runtime; you dereferenced it.

Fix: add an explicit loading-state branch BEFORE the data render branch. Don't trust ! across async boundaries.

Part 9 · Going further

You've got the system live. Here's what to add next, in priority order.

- **Morning briefing:** Telegram message at 7am with today's calendar, top 3 goals, yesterday's habit completion, finance change. Vercel cron + Telegram sendMessage API.
- **Voice journal:** long-form voice notes that go to a separate journal table (not the CRM). Same pipeline, different routing.
- **Sub-tasks:** a `parent_task_id` column on tasks lets you nest. Useful once you start using the CRM for projects.
- **Compress view:** a quarterly view that lists every task untouched in 30+ days and forces you to mark dead or revive.
- **Bulk select + undo toast:** CRM ergonomics. Multi-select tasks, mark all done with one undo. Saves clicks when you do a weekly sweep.
- **Cron-to-GitHub backup:** nightly cron exports your full database as JSON and pushes a commit to a private GitHub repo. Belt-and-suspenders against Supabase outages.
- **Public dashboard view:** a read-only version you can share with a partner / VA / family. Different password, RLS policies that filter to public-safe rows only.
- **Weight + body comp:** add to the Health tab. Daily weight log, rolling-7-day average, plotted over time.
- **Ask my OS:** the natural language Q&A that uses `memory_chunks` (see Part 6).
- **Multiple users:** if you ever want to ship this for friends. RLS policies, per-user data, separate Telegram bots. Architecture stays identical; you just generalise from `USER_ID = "miles"` to `auth.uid()`.

Appendix A · All copy-paste prompts

Every prompt block from the guide, listed in build order. Drop into Claude Code / Codex / Cursor in sequence.

A1 Project scaffold

PROMPT · paste into your AI coding agent

```
Create a new Next.js 15 project with the App Router and TypeScript strict mode. Set up Tailwind CSS with a dark default theme. Add a globals.css with oklch colour tokens. Initialise git.
```

A2 Database migration

PROMPT · paste into your AI coding agent

```
Create supabase/migrations/0001_init.sql with tables: entities, raw_captures, tasks, daily_logs (log_date NOT NULL), memory_chunks (with vector(1536) embedding column), audit_log. Add ivfflat index on memory_chunks.embedding. Enable RLS deny-all on all tables.
```

A3 Auth gate

PROMPT · paste into your AI coding agent

```
Add a single-password auth gate. HMAC-signed cookies. Env vars: AUTH_SECRET, DASHBOARD_PASSWORD. Files: middleware.ts, app/login/page.tsx, app/api/auth/login/route.ts, app/api/auth/logout/route.ts. Also support x-api-secret header for programmatic access.
```

A4 Port the design

PROMPT · paste into your AI coding agent

```
Port my Claude Design HTML mockup at [path] into Next.js components under components/dashboard/. One component per card. Shared Panel wrapper for glassmorphism. TopRail + Shell components. Placeholder data.
```

A5 Telegram capture pipeline

PROMPT · paste into your AI coding agent

```
Build app/api/telegram/webhook/route.ts. Verify secret token + user ID. Voice: Whisper transcribe. Text: pass to lib/router/classifyCapture.ts (Anthropic primary, OpenAI fallback, regex last-resort). Write to raw_captures, route to downstream table, embed to memory_chunks, audit_log. Reply with inline urgency keyboard.
```

A6 Web capture form

PROMPT · paste into your AI coding agent

Add a floating capture box at the bottom of the dashboard. POSTs to `/api/capture` which runs the same pipeline as the Telegram webhook.

A7 Operator + Session cards

PROMPT · paste into your AI coding agent

Build Operator (static) and Session (top 3 key tasks today) cards. Session reads `/api/tasks?status=open`, filters `urgency=today AND key=true`, ranks by `priority_score`.

A8 Calendar card

PROMPT · paste into your AI coding agent

Build a Calendar card. Use `ical.js` (NOT `node-ical`). Read `GOOGLE_CALENDAR_ICAL_URL`. Expand recurring events with `event.iterator()` over a 14-day window. 7-day strip, click-day-to-view. Auto-scroll NOW marker. Module-memory parse cache 5min. Response cache-control: no-store.

A9 Habit Tracker

PROMPT · paste into your AI coding agent

Build Habit Tracker. State in `daily_logs.notes` JSON. `localStorage` cache + Supabase sync via POST `/api/habits/[date]`. Use `localStorageKey()` – user's clock, not server's.

A10 CRM

PROMPT · paste into your AI coding agent

Build full CRM. 4 urgency tiers. 3 views (Smart / Kanban / Category). Drag-drop reorder persists `priority_score`. Click task → side drawer. New tasks insert at top of tier. Persist view selection to `localStorage`. API: GET/POST/PATCH/DELETE `/api/tasks`, POST `/api/tasks/smart` for NL search via Claude.

A11 Nutrition

PROMPT · paste into your AI coding agent

Build Nutrition. Two API routes: `/estimate` (text → kcal+macros) and `/redistribute` (name+kcal → macros). Bidirectional coupling. Date-scoped storage key for daily reset.

A12 Health tab

PROMPT · paste into your AI coding agent

Build Health tab. Read `/api/nutrition?days=30`. Render daily table with `expand-to-meals` on click. Average over logged days only.

A13 Goals

PROMPT · paste into your AI coding agent

Build Goals card. Two stacked sections (Week + Month). Sentinel anchor date so they never auto-clear. Schema: `daily_logs.notes.goals_week_items` and `goals_month_items`.

A14 Finance Pulse

PROMPT · paste into your AI coding agent

Build Finance Pulse with AI extraction. Service account auth. Drive API `export-to-XLSX`. `exceljs` parse. Send all tabs to Claude. Save snapshot to `daily_logs.notes.finance`. Page loads NEVER trigger AI – only manual ☐ or cron. Vercel cron daily at 5am UTC.

A15 Memory layer

PROMPT · paste into your AI coding agent

Add memory layer. On every text write, also write to `memory_chunks` with OpenAI `text-embedding-3-small`. Add POST `/api/memory/search`. Add Brain tab as a search bar over memory.

A16 Ask endpoint

PROMPT · paste into your AI coding agent

Add `/ask` endpoint. Embed question, top-20 nearest chunks, send to Claude with strict source-citing system prompt. Stream response.

A17 Demo mode

PROMPT · paste into your AI coding agent

Add a demo mode toggle to the top rail. When on, every card swaps to fake-but-realistic data from a `lib/demoData.ts` factory. CRM is fully isolated – adds in demo never touch the real DB. Synthetic API responses. `localStorage` persists demo state.

Appendix B · Env vars checklist

Every env var the build uses. Push all of these to Vercel before going to production.

Variable	Where it comes from	Used by
NEXT_PUBLIC_SUPABASE_URL	Supabase dashboard	Frontend + server
NEXT_PUBLIC_SUPABASE_ANON_KEY	Supabase dashboard	Frontend
SUPABASE_SERVICE_ROLE_KEY	Supabase dashboard	Server only
ANTHROPIC_API_KEY	console.anthropic.com	Classifier, finance, journal
ANTHROPIC_MODEL	static — e.g. claude-sonnet-4-6	Classifier
OPENAI_API_KEY	platform.openai.com	Whisper, embeddings, fallback classifier
OPENAI_CLASSIFIER_MODEL	static — e.g. gpt-4o-mini	Fallback classifier
AUTH_SECRET	random 32-byte hex (openssl rand -hex 32)	Auth gate
DASHBOARD_PASSWORD	memorable string you choose	Auth gate
API_SECRET	random hex (for programmatic access)	API routes
TELEGRAM_BOT_TOKEN	BotFather	Capture
TELEGRAM_WEBHOOK_SECRET	random hex	Capture
TELEGRAM_USER_ID	@userinfobot	Capture
GOOGLE_CALENDAR_ICAL_URL	Google Calendar settings	Calendar card
GOOGLE_SHEETS_FINANCE_ID	sheet URL between /d/ and /edit	Finance Pulse
GOOGLE_SERVICE_ACCOUNT_EMAIL	JSON key from Cloud Console	Finance Pulse
GOOGLE_SERVICE_ACCOUNT_KEY	private_key field from JSON	Finance Pulse
CRON_SECRET	random hex	Vercel cron auth
USER_TIMEZONE	e.g. Europe/Madrid	Day-rollover helpers
USER_ID	static string (e.g. "miles")	All routes

Final note. The system is genuinely never “done.” Every week of using it surfaces another small thing you want it to do. Fix those one at a time. After three months the dashboard knows you better than any productivity app ever could — because you literally built it around how you actually work.

Ship it. Use it. Iterate.

— Miles Deutscher / AI Edge